

OpenJev: Typed Decisions from Small Models

What it takes to make a 1.7B decoder, or a 150M encoder,
answer many typed questions about one input in a single forward pass

Prathamesh Saraf
github.com/S1LV3RJ1NX/openjev

September 2026

Abstract

Many production decisions are not open-ended generation but a fixed menu: route this message to one of 77 intents, flag whether it mentions clinical risk, rate its urgency. Jev, a commercial API from TypeSafe, makes exactly this the primitive, and does it well: it answers many typed questions about one input in a single call, and on a 77-way intent task it has never seen it scores roughly 0.863. This paper asks what it takes to build an open equivalent, and reports honestly where that succeeds and where it does not.

What we built. OpenJev encodes the state once as a shared prefix, places every candidate option after it behind a marker token, isolates questions from one another with block-diagonal attention, and normalises scores within each question’s own option set. Ten questions therefore cost one forward pass and 6% more latency than one, and the output is a calibrated probability over a closed menu rather than generated text. The system we recommend, and the one every headline number here refers to, is rank-16 LoRA adapters on a 1.7B decoder backbone, merged into the frozen weights at load. A 150M bidirectional encoder runs the identical format and is retained as the documented alternative and as the control in our architecture ablation. We release the code, two adapters, two encoder checkpoints, and three datasets including a 279-task, 323,466-row training mixture.

Held-out transfer. On seven public tasks never trained on, 600 rows each, every task scored at its full advertised menu, the LoRA decoder reaches **29.6× chance**: a 151-way menu at 0.702 (106.0×), a 60-way at 0.775, a 77-way at 0.605, and every task clearing both $1/K$ and its majority-class baseline. Trained on identical data and scored on identical items, the encoder reaches **17.2×**, loses on all seven tasks, and leaves the ordinal one level with its majority baseline. That ablation settles the architecture, since merging the adapter removes the latency argument for the encoder (1.07×).

The main finding is negative, and it is the useful one. The architecture does not generalize on its own. A strong bidirectional encoder with this exact output format, untrained, scores **0.7× chance** on held-out schemas, and a 61-task training mixture leaves it at 0.9×. Generalization is instead a property of the training data. Padding training menus with borrowed distractor labels, together with a mixture rebuild in the same change, moves a 77-way held-out menu from a literal 0.000 to 0.205; a model that has never seen a large menu collapses onto one label when handed one, though we did not separate the two interventions and so cannot attribute the gain cleanly. The backbone swap moves the suite from 17.2× to 29.6×; the data work is the difference between nothing and a working system on either backbone.

Both headline figures are corrections. Auditing our own results against a clean clone found that at the context budget we had been using, the packer silently dropped a 151-option menu to 64 while the multiple of chance was still computed against 1/151. The decoder’s 32.8× became 29.6× and the encoder’s 21.9× became 17.2×, Banking77 fell from 0.737 to 0.605, and `clinc_oos` fell from 0.783 to 0.702. The encoder figure additionally came from an in-training evaluator on a superseded version of the suite and did not reproduce. We report the withdrawn numbers alongside the corrected ones.

Against Jev, the result splits cleanly on one axis: labels. Given 395 labelled examples and 258 seconds, a rank-16 LoRA adapter on Qwen3-1.7B scores intent 0.979 against Jev’s 0.941 ($p = 9.8\text{e-}04$) and multi-label exact set 0.909 against 0.822 ($p = 7.2\text{e-}6$) on a hardened healthcare routing benchmark: three wins, four statistical ties, no losses. **The comparison is asymmetric and in our favour**, since we are fine-tuned on that task and Jev is zero-shot on it, so the claim it supports is that a few hundred labels are worth more than the gap between an open 1.7B model and a closed API. The adapter is 87 MB and outperforms opening all 1,725M parameters (0.929, 3.4 GB). Curiously, initialising that adapter from our general checkpoint rather than from base weights makes it *worse* (0.953, $p = 0.012$), reversing a +36 point gain from the same intervention on the encoder: staged fine-tuning pays only when the base model cannot perform the task from at all. Without labels the picture inverts. Measured over the whole held-out suite rather than the single task we had been quoting, Jev wins five of seven and ties two, 38.3× chance against our 29.6×, including 0.938 against 0.702 on the 151-way menu. Applied zero-shot to the routing task the decoder scores intent 0.467 against the encoder’s 0.601, so the benchmark ordering does not survive contact with a deployment task either.

Takeaway. Use Jev, or something like it, when you need typed decisions on a schema you have no labels for; that capability is real and we did not reproduce it. Use OpenJev when you can label a few hundred examples of your actual task, want the model on your own hardware, need full-precision probabilities and determinism, or need multi-label routing where a single choice cannot express the answer. Within OpenJev, take the merged LoRA decoder unless a p95 latency budget near 20 ms or a 0.6 GB footprint rules it out, in which case the encoder is the documented fallback at a measured cost in accuracy. Benchmark transfer and deployment readiness are not the same thing, and we report the gap between them rather than the better number. All claims are reproducible from a public repository, with negative and retracted results given the same prominence as the positive ones.

1 Introduction

Consider a pharmacy support queue. A message arrives: *“I need a refill on my thyroid tablets, and are you open on Sunday? Also I’ve been getting dizzy since starting them.”* A system routing this must decide, simultaneously, which topics it concerns (two), whether it mentions clinical risk that a human should see (it does), and how urgent it is. Each of these is a decision over a *known, closed* set of answers.

The dominant approach is to ask a language model and parse what comes back. This works, and it is expensive in three distinct ways. Each question costs its own generation. The output is unconstrained, so it must be coerced onto the menu by grammar-constrained decoding or by retrying. And the resulting token probabilities are a poor basis for the thresholding that operational systems need, because they describe a sampling distribution over strings rather than a calibrated belief over options.

Readers new to this area: Appendix A defines every technical term used here, from “token” through to the statistical tests, so the main text can stay short.

1.1 Typed decisions

We use *typed decision* for a question whose answer set is fixed in advance and known to the system. Three types cover most of what production routing and guardrail systems need:

- **choice**: exactly one of K options. Intent, category, routing target. K reaches 151 in our evaluation.
- **score**: one level on an *ordered* scale. Severity, urgency, quality. The ordering is information a **choice** discards.
- **noul**: yes or no. Flags and safety gates. Several asked together express multi-label structure, which a single **choice** cannot: a softmax over intents has no way to say “both”.

1.2 Jev, and why it is the right thing to measure against

The system that made this concrete for us is **Jev**, a commercial typed-decision API from TypeSafe. It is closed, and it is good. Rather than treat it as a competitor to be beaten on a leaderboard, we treat it as an existence proof and a measuring stick, and it deserves credit on both counts.

Three things Jev gets right, which we verified by measurement rather than taking on faith:

- **It asks the right question.** Most of the industry responds to structured-output needs by constraining a generative decoder. Jev instead makes the typed decision the primitive. The API accepts a state and a set of typed questions and returns probabilities over fixed menus. Having used it, we find this to be the correct abstraction for routing and guardrail work, and the framing of this paper is borrowed from it.
- **The shared-prefix design pays off, and we can see it in the timings.** Latency is flat from 2 to 255 options, and an eight-question call costs the same wall time as a one-question call (392 ms against 407 ms). Adding 253 options costs roughly 8,200 input tokens and approximately zero milliseconds. That is a real engineering result and it is what convinced us the architecture in §3 was worth building.
- **Its zero-shot ability is the genuinely hard part, and it is ahead of ours on every task we measured.** Across the seven held-out schemas it wins five and ties two, 38.3× chance against our 29.6×. On Banking77, a 77-way intent task, it scores 0.863 where our best equivalent, the LoRA decoder, reaches 0.605; our encoder reaches 0.343. After considerable effort we did not close this gap, and §6.8 shows the gap is wider still on a realistic deployment task. If you need typed decisions on a schema you have no labels for, Jev is the better tool and this paper does not argue otherwise.

We also record where we found its limits, because a measuring stick should be described accurately in both directions: probabilities are quantised to a 0.01 grid with 71.9% of returned values at a hard zero, which removes log-odds and tail-risk reasoning; there is no seed, so results are not reproducible run to run; and on our hardened routing benchmark it misses roughly one in five obliquely-worded clinical risks.

1.3 What this paper is about

OpenJev is an open implementation of an architecture for answering many typed questions about one input at once, and an empirical account of what makes it work. We reverse-engineered the

observable behaviour of Jev, built an architecture consistent with it, and then measured which parts of the recipe actually carry the performance. The interesting result is not the architecture, which is a recombination of published ideas (§2). It is that almost none of the performance comes from it.

The anchor system, and the alternative we kept. Every headline number in this paper comes from rank-16 LoRA adapters on a 1.7B decoder backbone (Qwen3-1.7B), merged into the frozen weights at load. We also built and trained the identical output format on a 150M bidirectional encoder (ModernBERT-base), and ran the two against each other on the same data and the same evaluation items. The decoder wins every one of the seven held-out tasks and takes the suite mean from $17.2\times$ chance to $29.6\times$, so the decoder is the recommended artifact. The encoder is not discarded. It remains the right choice under a tight p95 latency budget or a small footprint (§6.7), and it is the control that makes several findings below legible at all: an intervention that helps one backbone and hurts the other is only visible if you have both (§7.4).

Four findings organise the paper.

The architecture does not generalize on its own. A pretrained bidirectional encoder, given the option-scoring output format and no task training, scores $0.7\times$ chance on held-out schemas, worse than guessing. Its larger sibling reaches $2.2\times$. A pretrained decoder given the same format scores $0.8\times$ with options concatenated plainly, and $11.5\times$ once each option is framed as a yes/no question under a task preamble. The output format makes zero-shot scoring *possible*; it does not make it *present* (§6.3).

Generalization is a property of the training mixture, and some of it is synthesizable. Our first mixture left held-out transfer at $0.9\times$ chance. The single largest fix was not more data but a different *shape* of data: padding training menus with distractor labels borrowed from other tasks, which moved a 77-way held-out menu from a literal 0.000 to 0.205. A model that has never seen a large menu collapses onto one label when handed one, and you can manufacture large menus without finding datasets that have them (§6.4).

The backbone choice is real, and the benchmarks that settle it do not predict deployment. Holding data and evaluation items fixed, the decoder beats the encoder on all seven held-out tasks. Applied zero-shot to the routing task, the same decoder is *worse* than the encoder (intent 0.467 against 0.601). Both are unusable there, so the architecture call stands, but $29.6\times$ is a benchmark number and not a readiness claim (§6.8).

With labels, small models win; without them, they do not. A rank-16 adapter on a 1.7B backbone, trained on 395 examples, beats the commercial API on three of seven router measures and ties the rest. The same architecture without task labels remains far behind it. That split is the honest summary of where this technology stands (§6.5, §8).

1.4 Contributions

1. An architecture and open implementation for multi-question typed decisions in a single forward pass, with latency flat in the number of questions (10 questions cost 6% more than 1) and full-precision calibrated probabilities, released as a merged rank-16 LoRA decoder with a bidirectional-encoder variant of the same format.

2. A controlled architecture ablation between the two backbones on identical data and identical items, which the decoder wins on all seven held-out tasks ($17.2\times \rightarrow 29.6\times$ chance), together with the latency and footprint measurements that say when the encoder is still the right answer.
3. *Label-space augmentation*, a training-time intervention that manufactures the menu-size diversity a mixture lacks, and which is the difference between no transfer and useful transfer at high cardinality.
4. A measured account of parameter-efficient fine-tuning for this task shape, including the result that a 17M-parameter adapter outperforms opening all 1,725M parameters, at 1/39 the artifact size.
5. A reproducible benchmark suite with an enforced contamination guard, a held-in control that distinguishes “no transfer” from “a bug”, and a hardened routing task built to retain headroom where the reference system saturated.
6. A set of negative and retracted results reported at the same prominence as the positive ones (§7), including two interventions that made things worse and one conclusion we withdrew after finding the bug that produced it.

Everything is reproducible from a public repository, with released checkpoints, adapters, and datasets listed in Appendix B.

2 Background and related work

Every structural idea in OpenJev exists in the literature. We state the provenance plainly, because the paper’s contribution is empirical rather than architectural.

Scoring options instead of generating them. UniMC [11] recasts multiple-choice NLU by inserting an option-marker token before each candidate and reading a decision from the encoder’s state at that position, avoiding generation entirely. Our readout is the same idea. The consequence people usually reach for, that such a model “cannot hallucinate”, is narrower than it sounds: the model cannot emit a string outside the menu, which says nothing about whether the option it picks is right.

Encoding a shared prefix once. Parallel Context Windows [5] and related work formalise attention masks in which several segments each attend to a common prefix and to themselves, but not to one another. We use exactly this to isolate questions, so that an answer does not depend on which other questions were asked in the same call. On a causal backbone the prefix is ordinary left-context, which is what a KV cache is for.

Multi-task training for zero-shot transfer. T0 [6] and the Flan collection [3] establish that training across many tasks produces zero-shot ability on unseen ones, and that the gain continues accruing with task count into the high hundreds. Our mixture is assembled from **tasksource** [8], which normalises a large number of classification and multiple-choice datasets into a common interface. Flan’s task-count finding is the reason we treat 279 tasks as a floor rather than a target.

Intermediate-task transfer. Training on an intermediate task before the target is STILTs [4]; the domain-adaptive variant is Gururangan et al. [1]. Both report that the gain is largest when the target dataset is small, and that it is inconsistent across task pairs and can be negative. We observe both effects: a large gain at 395 target examples (§6.5), and two mixture changes that made held-out transfer *worse* (§7).

Parameter-efficient adaptation. LoRA [2] trains low-rank updates beside frozen weights. Our use is conventional; the finding that it *outperforms* full fine-tuning here, rather than merely approximating it cheaply, is reported in §6.5 together with the most likely explanation.

Ordinal and rejection heads. CORN/CORAL [7] give rank-consistent ordinal objectives, and adaptive decision boundaries [12] detect out-of-scope inputs without negative samples. We implement neither; **score** is currently a softmax over levels whose order is used only in rendering and in the expectation-based readout. This is a limitation and we mark it as such (§8).

The system we compare against. Our reference is a commercial typed-decision API, `jev-1.13.0` from TypeSafe, measured by black-box probing on 20 September 2026 at the sample sizes stated. We inferred its design from behaviour: latency is flat from 2 to 255 options and an eight-question call costs the same wall time as a one-question call, which is the signature of a shared prefix with parallel readout. All figures attributed to it are one-day measurements of a hosted endpoint from one network location, and should be reproduced before being relied on.

3 Architecture

3.1 Problem statement

An instance is a state s and a set of questions $Q = \{q_1, \dots, q_m\}$. Each question q_i carries instructions and an option set $O_i = \{o_{i,1}, \dots, o_{i,K_i}\}$, with K_i varying freely across questions in the same call. The model must return, for every question, a distribution over that question’s own options:

$$p(o_{i,j} \mid s, q_i), \quad \sum_{j=1}^{K_i} p(o_{i,j} \mid s, q_i) = 1.$$

Two properties are required. The cost must be sublinear in m , since a production router asks ten questions where a benchmark asks one. And $p(\cdot)$ must be a usable probability, because the operational decision is a threshold on it.

3.2 The system, and the one variant of it

The deployed system is a causal decoder backbone, Qwen3-1.7B, adapted with rank-16 LoRA and merged into the frozen weights at load. The five components below are packing, block-diagonal attention, marker readout, grouped log-softmax, and the adapter. Everything in this section describes that system.

One component, the backbone, has a second implementation. The same packing, the same mask, the same grouped normalisation and the same loss run on a 150M bidirectional encoder, ModernBERT-base, with a linear scorer in place of the vocabulary-logit readout. We describe that variant alongside each component it changes, because it is a released artifact and because the architecture ablation in §6.1 is only interpretable if both paths are specified. It is the alternative, not the default: its case is latency and footprint (§6.7), not accuracy.

block 0: shared state	block 1	block 2	block 3
<i>"I need a refill, and are you open Sunday?"</i>	intent? refill • hours •	clinical? no • yes •	urgent? low • high •

Figure 1: Packing. The state is encoded once. Each option ends in a marker • where the score is read. Every block attends to block 0 and to itself; no block attends to another. Asking question 3 therefore cannot change the answer to question 1.

3.3 Packing

We write the state once, then each question and its options after it, as a single token sequence (Figure 1); which positions may then attend to which is Figure 2. Each option is followed by a *marker* token, and the model’s output for that option is read at the marker position. On a causal backbone the marker must follow the option, since it has to have seen it; on the bidirectional encoder variant it may equally precede the option, which is how UniMC writes it [11].

Formally, the packed sequence carries a block assignment $b(t) \in \{0, 1, \dots, m\}$ for each token position t , where block 0 is the shared state prefix and block i holds question q_i with its options.

3.4 Block-diagonal attention

Let $A(t, u) \in \{0, 1\}$ indicate whether query position t may attend to key position u . We impose

$$A(t, u) = \underbrace{\mathbb{1}[b(u) = 0 \vee b(u) = b(t)]}_{\text{our rule}} \cdot \underbrace{C(t, u)}_{\text{backbone's own}}, \quad (1)$$

where $C(t, u) = \mathbb{1}[u \leq t]$ for the causal decoder and $C(t, u) = 1$ for the bidirectional encoder variant. Both factors are indicators, so the product is the conjunction.

The first factor is what makes the shared prefix legitimate: every question sees the full state. The second is what makes the result well defined. Without it, question 2 attends to question 1 and an answer becomes a function of which other questions happened to be asked in the same call. On the decoder the state already precedes every question, so the rule’s only remaining work is isolating question blocks from each other, and the prefix is ordinary left context that a KV cache already knows how to reuse. Figure 2 shows both cases.

The isolation is measured, not assumed. A block-diagonal mask is easy to write and easy to get subtly wrong, and a wrong one fails silently: questions leak into each other and the model still returns plausible answers. We tested it adversarially. Adding ten sibling questions that explicitly assert the answer to the target question moved the target’s probability vector by 0.0022 mean absolute deviation, against a replication noise floor of 0.0029 measured by re-running the identical input. The perturbation is smaller than the noise, with zero argmax flips in thirty trials. One question cannot prompt-inject another, which is the property the mask exists to provide and the one a packed design is most likely to lose.

Memory. The mask is $O(n^2)$ in sequence length, which is the usual attention cost and not specific to this design, but it does bound how many questions fit in one call. At our 2048-token budget a ten-question router with 24 options packs comfortably; a 151-option menu rendered with long descriptions does not, and the implementation degrades by subsampling the menu (always keeping the gold) rather than failing. This graceful degradation is also what produced the largest

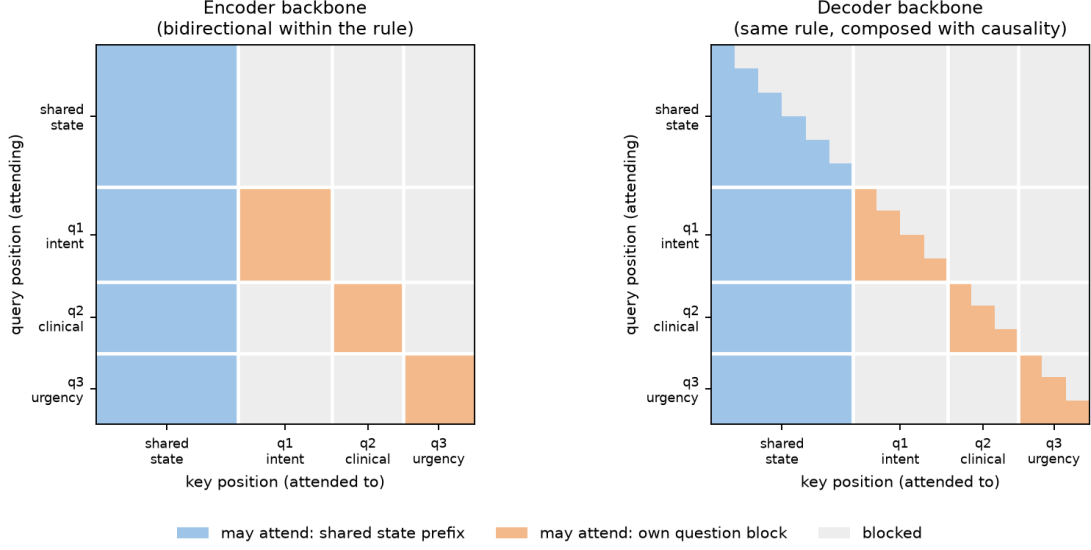


Figure 2: The mask of Eq. 1 for a packed sequence with a shared state and three questions. Blue is attention to the shared prefix, orange is attention within a question’s own block, grey is blocked. No question can see another, so its answer does not depend on what else was asked. Right, the deployed case: the same rule composed with causality on the decoder backbone. Left: the encoder variant, where the rule is the only constraint.

measurement fault in this work, because for some time the evaluator scored against the menu it had asked for rather than the menu the model was shown (§6.1). The decoder’s per-option template costs more tokens than the encoder’s, so it clips sooner at a given budget: a full 151-way menu needs 6144 tokens on the decoder path.

3.5 Readout and grouped normalisation

Let h_t be the final hidden state at marker position t , and let $\mathcal{M}_i$ be the marker positions of question q_i . A scalar score is produced per option. On the decoder this is the difference of two vocabulary logits at the marker,

$$z_t = [W_{\text{lm}}h_t]_{\text{yes}} - [W_{\text{lm}}h_t]_{\text{no}}, \quad (2)$$

optionally plus a residual head $g(h_t)$ initialised to zero, so that training begins at exactly the untrained readout. Reading through the pretrained language-modelling head is what lets an untrained decoder score a menu at all, and §6.3 measures how much of the untrained ability that choice is responsible for. On the encoder variant there is no vocabulary head to borrow, so the score is a shared linear scorer over the marker state, $z_t = w^\top h_t + b$, trained from random initialisation.

Because K_i differs across questions packed into one sequence, scores are normalised *within* each question rather than globally:

$$p(o_{i,j} \mid s, q_i) = \frac{\exp(z_{t_{i,j}})}{\sum_{t \in \mathcal{M}_i} \exp(z_t)}. \quad (3)$$

This is a segmented softmax over a ragged partition of the flat marker array. Writing y_i for the index of the gold option of question q_i , and $\mathcal{A}$ for the questions this example actually answers (a

Algorithm 1 Forward pass for one packed instance

Require: state s , questions $Q = \{q_i\}$ with option sets O_i

- 1: $x, b, \mathcal{M} \leftarrow \text{PACK}(s, Q)$ ▷ tokens, block ids, marker positions
 - 2: $A \leftarrow \text{mask from Eq. 1 using } b \text{ and backbone constraint } C$
 - 3: $H \leftarrow \text{BACKBONE}(x, A)$ ▷ one forward pass, all questions
 - 4: $z \leftarrow \text{SCORE}(H[\mathcal{M}])$ ▷ Eq. 2, or linear for the encoder
 - 5: **for all** questions q_i **do**
 - 6: $p_i \leftarrow \text{SOFTMAX}(z[\mathcal{M}_i])$ ▷ grouped, Eq. 3
 - 7: **end for**
 - 8: **return** $\{p_i\}$
-

row may supervise only some of them), the loss is

$$\mathcal{L} = -\frac{1}{|\mathcal{A}|} \sum_{i \in \mathcal{A}} \log p(o_{i,y_i} \mid s, q_i). \quad (4)$$

A worked example. Suppose one packed sequence holds q_1 with $K_1=3$ and q_2 with $K_2=2$, so the flat marker array has five entries with scores $z = [2.0, 1.0, 0.5, 0.2, 1.4]$ and the partition $\mathcal{M}_1 = \{1, 2, 3\}$, $\mathcal{M}_2 = \{4, 5\}$. A global softmax over all five would make q_1 's options compete with q_2 's, which is meaningless. Grouped, we get $p_1 \approx [0.63, 0.23, 0.14]$ and $p_2 \approx [0.23, 0.77]$, each summing to one. If the golds are $y_1=1$ and $y_2=2$, then $\mathcal{L} = -\frac{1}{2}(\log 0.63 + \log 0.77) \approx 0.36$.

3.6 Rendering the three primitives

How an option is written to the sequence matters more than one would expect, and §6.3 quantifies it. The rules we converged on:

- **choice:** label and description both, since both carry meaning. Where the two are identical, as in multiple-choice tasks whose option *is* its own description, they are collapsed, or every option is emitted twice.
- **score:** the level description alone. Rendering a level as "0: very negative: ..." asks the model to score a bare index.
- **noul:** bare polarity, **no/yes**, and **not** the descriptions. A **noul**'s two descriptions necessarily restate one judgement from opposite sides, so scoring both measures their overlap. Measured, this costs AUROC 0.63 against 0.712 (§6.3).

3.7 Adapters and merging

The decoder is not fine-tuned by opening its weights. We train low-rank updates beside them [2]: for a frozen projection $W_0 \in \mathbb{R}^{d \times k}$, the forward pass computes $W_0 x + \frac{\alpha}{r} B A x$ with $B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times k}$ and $r = 16$, $\alpha = 32$, applied to the query, key, value, output, gate, up and down projections at every layer. That is 17M trainable parameters against 1,725M frozen ones, and an artifact of 87 MB rather than 3.4 GB. §6.5 reports that this outperforms opening all 1,725M parameters on our target task, which is not the usual expectation for a parameter-efficient method.

Two consequences matter for the architecture rather than for training.

Merging. Since BA is linear and sits in parallel with W_0 , a trained adapter can be folded in once, $W \leftarrow W_0 + \frac{\alpha}{r}BA$, after which inference runs the base graph exactly and costs nothing extra. Left unmerged it is an additional matmul per target module per layer, which we measure at $1.96\times$ p50 latency against $1.07\times$ merged (§6.7). That is close to a doubling of latency for no change in output, so the released loader merges on construction and merging is not an optimisation a user has to remember. It is also the fact that removes the latency argument for the encoder backbone: a model eleven times larger runs 7% slower once the adapter is folded in.

One backbone, many tasks. Because the adapter is small and separable, a single copy of the base weights can serve many typed-decision schemas by swapping 87MB. Our general mixture adapter and our router adapter are both of this form. They are not composed: §7.4 reports that initialising the second from the first is measurably worse than initialising from base.

3.8 Cost

The state is encoded once regardless of m , so the marginal cost of a question is its own options. Measured on the ten-question router with a 1.7B backbone, median latency per state is 69ms for one question and 74ms for ten: **ten questions cost 6% more than one**. The reference API shows the same signature, 392ms against 407ms, which is the evidence that it shares this design.

Against the obvious alternative. The alternative to packing is one sequence per question, each carrying its own copy of the state, run as a batch. Table 1 measures both on ModernBERT-base at 50 questions.

state	packed tok	naive tok	packed	naive	speedup
~90 tok	713	3,800	9.6 ms	9.6 ms	1.00×
~700 tok	1,133	24,800	9.7 ms	51.7 ms	5.33×
~2.8k tok	2,573	96,800	11.0 ms	263.5 ms	23.98×

Table 1: Packing against one sequence per question, median of ten runs after warmup, batch size 1. The saving is in the state, so it grows with state length and is nothing at all when the state is short.

The honest reading is the first row. **At short states packing buys nothing**, because 9.6ms is a fixed-overhead floor that both paths hit. The technique pays when the state is long enough that re-encoding it dominates, which is the regime this design targets and not a universal claim. Latency is also flat in question count: at a 700-token state, one question and fifty both take 9.6ms.

On the two sets of latency numbers in this paper. An earlier draft reported two sets that disagreed, 69 to 74ms here against 37.7 to 55.9ms in §6.7, because the first pair was taken while a training run shared the GPU. Re-measured on an idle device the decoder is 22.7ms at p50 and 55.5ms at p95, matching the lighter-load set to within 0.3ms. The contended figures are withdrawn and the ratios above are recomputed from the idle measurement.

4 Data

Since our central claim is that generalization lives in the training data rather than the architecture, the data deserves the same scrutiny usually reserved for the model.

4.1 Training mixture

We assemble 279 tasks and 323,466 rows from `tasksource` [8], normalised into the typed-decision format. Composition after cleaning:

primitive	tasks	notes
<code>choice</code>	234	option counts 2 to 174; 80 tasks have per-example menus
<code>noul</code>	34	
<code>score</code>	11	mostly three-level sentiment; see §8
total	279	323,466 rows

The lopsidedness is a limitation, not a design choice, and it predicts the result pattern: `choice` transfers best and `score` worst.

Per-example option menus. 80 tasks come from the multiple-choice family, where each row carries its own answer set rather than sharing a fixed menu. Supporting this in the trainer is what unlocked that family, which is 261 of the 668 tasks `tasksource` exposes.

4.2 Contamination guard

A held-out suite is worthless if the training mixture contains the same data under a different name, and aliasing is the normal case: `tasksource` calls one dataset `banking77`, while the Hub also carries `PolyAI/banking77` and `mteb/banking77`.

Each evaluation task therefore declares every alias of its source, a manifest holds the union, and a matcher checks full names, owner-stripped basenames, and glob patterns rather than performing a set intersection. The check runs before every training run and refuses to proceed on a hit; it excluded 23 tasks when building this mixture, and caught a real leak during development. We verify it with a positive control, injecting `banking77` into a clean mixture and confirming rejection, since a guard that has never fired is indistinguishable from a guard that cannot.

4.3 Auditing, and what it found

Training is the expensive step, so faults that survive it and surface hours later as a bad number are worth catching first. We audit for gold answers outside their own menu, `score` golds stored as booleans (in Python `True == 1`, which silently rewrites levels 0 and 1), class imbalance too severe to learn, near-duplicate `noul` descriptions, split leakage, and contradictory labels.

Three findings changed the data materially.

The gold answer was always first. `tasksource` ships multiple-choice tasks correct-answer-first. Measured after ingestion, the gold sat at index 0 in **100% of rows across all 83 ingested tasks**. Stored that way, the data teaches “pick the first option” to anything reading the menu as written. We shuffle at ingestion so the stored data is honest, and drop tasks whose gold remains at a fixed index or is a single constant string.

Contradictory labels. 1,994 rows carried the same state under the same menu with different answers; whichever the model predicts, some are wrong, so the gradient is noise. One task contributed 195.

Algorithm 2 Label-space augmentation for one choice question

Require: menu O with gold y , distractor pool D , probability π , cap M

```
1: if UNIFORM(0, 1)  $\geq \pi$  then return  $O$ 
2: end if
3:  $n \leftarrow \lfloor \exp(\text{UNIFORM}(\ln(|O|+1), \ln M)) \rfloor$   $\triangleright$  log-uniform target size
4:  $E \leftarrow \emptyset$ 
5: while  $|O| + |E| < n$  and attempts remain do
6:    $d \sim D$ 
7:   if  $d \notin O$  and  $d \notin E$  then  $\triangleright$  never add a plausibly correct label
8:      $E \leftarrow E \cup \{d\}$ 
9:   end if
10: end while
11: return SHUFFLE( $O \cup E$ )  $\triangleright y$  still present and correct
```

Menus too long to pack. 1,787 rows had option sets exceeding the context budget on their own, some options being multi-paragraph passages.

Finally, we sweep every example through the same dataset object the trainer uses, including augmentation, before training starts: 323,466 examples verified in 227 seconds of CPU. Two earlier runs died on packing errors after twenty minutes and ten thousand steps respectively, and both faults were findable in advance.

4.4 Label-space augmentation

Our mixture’s option counts topped out near 20 while the evaluation suite reaches 151. A model that has never seen a large menu collapses onto one label when handed one, which is what an exact 0.000 looks like: chance alone would have produced roughly three correct in 200.

The intervention is to manufacture the missing shape. With probability π , a training menu is padded with labels borrowed from other tasks in the mixture, sampling a target size log-uniformly so small menus stay common and large ones appear often enough to matter. The gold answer is unchanged and still correct, so the example remains valid, but the model must now discriminate against a large menu.

Option order is shuffled every epoch for a second reason: on the reference system we measured the argmax flipping across option orderings on 7.5% of confusable items, at $2.8\times$ the resampling floor. That is per-item instability rather than a positional prior, so it cannot be corrected post hoc and must be trained out.

4.5 Evaluation suites

Held-out suite (Study G). Seven public tasks, 600 rows each, never trained on: `clinc_oos` ($K=151$), `banking77` ($K=77$), `massive_intent` ($K=60$), `sst5` and `helpsteer (score,  $K=5$ )`, `ag_news` ($K=4$), and `civil_comments (noul)`. The high-cardinality end is the point: a 151-way menu is where models that look fine on four-way classification fail.

Every evaluation reports a *held-in control*: accuracy on tasks the model was trained on. If held-out is at chance and held-in is high, the model genuinely fails to transfer; if both are at chance, it is a bug. We also run a trivial three-way sanity task that any working harness must pass. This is not ceremony. The sanity check caught a real partially-loaded-model bug that had produced a plausible, and wrong, results table.

Healthcare router (Study S). A synthetic pharmacy routing task: 988 items across 18 tiers, 395 train and 450 test, with ten questions per item (one intent **choice**, five topic **nouls**, four safety gates).

Its first version was useless: the reference API scored a clean 1.000 on four of its tiers, which cannot distinguish any two systems. We rebuilt it targeting where that system actually failed: obliquely-worded clinical risk, three-intent compounds, negation and conditionals, near-miss out-of-scope, subtle prompt injection, and degraded voice-transcript text. Splits are disjoint by content rather than by row, so paraphrases and noisy variants of an item all stay in one split.

5 Experimental setup

5.1 Two studies

The experiments answer two different questions, and they disagree. Keeping them separate is what makes the disagreement legible rather than contradictory.

Study G (general capability). Train on the 279-task mixture, evaluate on seven tasks never trained on. *Does it work on a schema you have no labels for?* This is the harder claim and the reference system’s main advantage.

Study S (specific use case). Train on 395 examples of the healthcare router, evaluate on its 450-item test split. *Given labels for your actual problem, how good does it get?*

5.2 Backbones

	decoder (<i>primary</i>)	encoder (<i>alternative</i>)
model	Qwen3-1.7B	ModernBERT-base
parameters	1,725M	150M
attention	causal + block-diagonal	bidirectional + block-diagonal
readout	logit(yes) – logit(no)	linear at marker
adaptation	LoRA $r=16$, merged	full fine-tune

Every result is reported on both backbones wherever we ran both, because the comparison between them is itself one of the experiments. Where only one figure is given, it is the decoder.

“Decoder” names the attention pattern, not the usage: nothing is generated token by token, and inference is a single forward pass with a parallel read at marker positions.

5.3 Training configurations

Study G uses one epoch over the mixture, batch 8 (decoder) or 16 (encoder), context 2048, label-space augmentation at $\pi=0.5$ with cap $M=120$. Study S uses six epochs over 395 examples. Four adaptation regimes are compared in §6.5: decoder LoRA ($r=16$, $\alpha=32$, on all attention and MLP projections, $\text{lr} = 2 \times 10^{-4}$), decoder head-only, decoder full fine-tune, and encoder full fine-tune.

5.4 Metrics

Accuracy, macro-F1, and multi-label exact-set match, with bootstrap confidence intervals over examples. For comparability across tasks with very different K we report *multiple of chance*, $\text{acc} \cdot K$,

since raw accuracy is not comparable: 0.28 on a 77-way menu is strong and on a 4-way menu is worse than guessing.

Two metric choices are load-bearing.

AUROC on binary tasks. Argmax accuracy on a `noul` describes the 0.5 threshold as much as the model. A task can rank almost perfectly and still score exactly the class balance by putting all probability mass on one side. We report both, and the gap between them is a calibration result rather than a transfer result.

Chance is not always the right floor. A skewed task can be beaten by always predicting its most common label. Where that baseline exceeds $1/K$ we test against it explicitly, which changes one of our conclusions about the encoder (§7.1) and leaves one decoder result thinner than its multiple of chance suggests (§6.1).

Scoring at the menu the model was shown. A multiple of chance is only meaningful against the option set actually presented. Our evaluator now records the menu after packing, scores against that, and names any task whose menu was clipped. It did not always do this, and §6.1 reports what that cost us.

Paired comparison. Against the reference system we use exact McNemar on the same items, reporting b_{10} (we are right, they are not) and b_{01} . An aggregate delta cannot distinguish a real difference from two systems disagreeing equally in both directions.

5.5 Reproducibility

All runs are on one H100. The contamination guard runs and logs on every training run. Checkpoints record their backbone, adapter rank, prompt preamble, option template, and fitted temperatures, because the same weights scored under a different prompt are a different system. Artifacts are listed in Appendix B.

6 Results

6.1 Held-out transfer, and the backbone ablation it settles

Table 2 is the central result of Study G: the LoRA decoder trained on the audited 279-task mixture, evaluated on seven public tasks it was never trained on, every task scored at its full advertised menu. The encoder column is the same mixture, the same augmentation, the same evaluation items, and the same harness, with only the backbone and its readout changed. Harness sanity 1.000; held-in control 0.847 to 0.980 accuracy, 1.7 to $2.0\times$ chance, on five tasks the model was trained on, so a weak held-out number here would be a transfer result rather than a bug.

Three things in the table matter beyond the mean. The high-cardinality end is where the gap is widest: a 151-way menu at 0.702 against 0.382, and a 77-way at 0.605 against 0.343. **Every task beats its majority-class baseline**, not merely $1/K$, which the encoder did not manage: `civil_comments` and `sst5` clear theirs at $p < 10^{-15}$, and `helpsteer` reaches 0.273 against 0.233 at $p = 0.010$, a margin thin enough that a change in sampling could erase it. On the binary task we also report ranking rather than argmax alone, where `civil_comments` reaches AUROC 0.828.

OpenJev encoder and decoder against the commercial reference

The answer differs by question: they are better with no labels, we are better with a few hundred, and on serving cost it is not close.

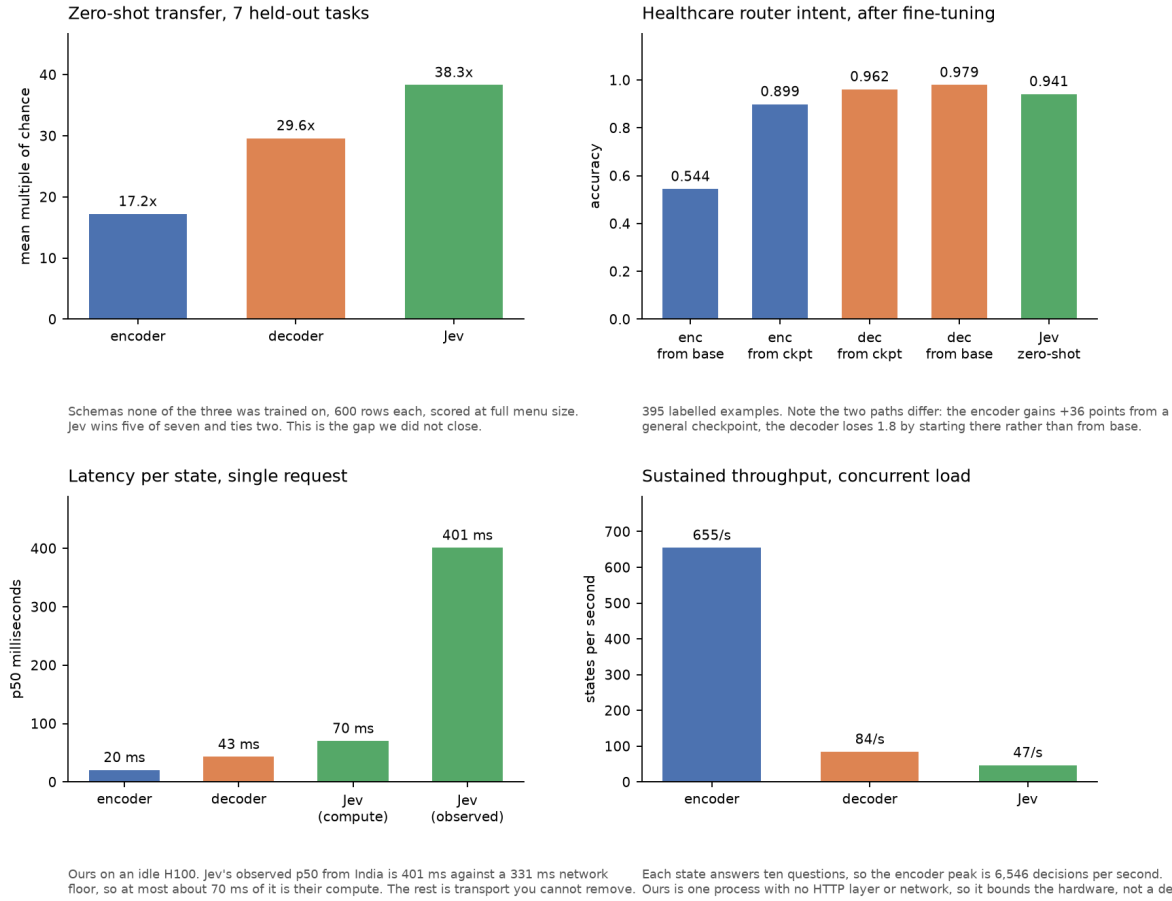


Figure 3: The two backbones and the commercial reference on the four questions a deployment actually asks. The reference system wins zero-shot transfer and we do not close that gap. Given a few hundred labels the ordering reverses, and on latency and sustained throughput it is not close. No single number summarises this, which is why we do not report one.

Banking77 zero-shot rises from 0.343 to 0.605 against the commercial reference’s 0.863. That closes part of the gap and leaves most of it. It is not parity and we do not claim it; Section 6.2 measures the same comparison on all seven tasks.

These numbers replace an earlier, inflated set. Auditing our own results against a clean clone of the public repository exposed a measurement fault. The packer drops options to fit `max_len`, always retaining the gold option, and our evaluation runs passed `--max-len 2048`. At that budget `clinc_oos` presented 64 of its 151 options and `banking77` presented 64 of 77, while the multiple of chance was still computed against $1/151$ and $1/77$. The accuracies were real; the multiples credited the model for a discrimination it never performed. We previously reported $32.8\times$ for the decoder and $21.9\times$ for the encoder; the corrected figures are $29.6\times$ and $17.2\times$, Banking77 falls from 0.737 to 0.605, and `clinc_oos` from 0.783 to 0.702. Correcting it took two attempts: at 4096 tokens banking77’s 77 options fit but `clinc_oos` is still clipped to 64, because the decoder’s per-option template costs tokens the encoder’s does not, and the full 151-way menu needs 6144. The encoder

task	K	chance	accuracy	95% CI	$\times$ chance	encoder
clinc_oos	151	0.007	0.702	[0.667, 0.738]	106.0	0.382
massive_intent	60	0.017	0.775	[0.742, 0.807]	46.5	0.473
banking77	77	0.013	0.605	[0.570, 0.642]	46.6	0.343
ag_news	4	0.250	0.793	[0.760, 0.828]	3.2	0.735
sst5	5	0.200	0.465	[0.425, 0.502]	2.3	0.412
civil_comments	2	0.500	0.742	[0.710, 0.777]	1.5	0.683
helpsteer	5	0.200	0.273	[0.240, 0.312]	1.4	0.262
mean multiple of chance					29.6	17.2

Table 2: Held-out transfer for the rank-16 LoRA decoder, 600 rows per task, never trained on, every task scored at its full advertised menu. The final column is the encoder variant on identical data and identical items (§7.1). Every task improves, and every interval excludes its chance floor. The suite began this work at $0.9\times$ chance.

figure additionally came from an in-training evaluator on a superseded version of the suite and did not reproduce. `eval_heldout.py` now records the menu the model actually saw, scores against that, prints $K \rightarrow k$ when they differ, and names every truncated task in a closing warning.

The architecture decision. Our ablation registry fixed three possible outcomes before this run so that the choice could not be rationalised afterwards. Latency does not separate the backbones once the adapter is merged ($1.07\times$, §6.7), and a 1.7B model fits a consumer GPU, so zero-shot transfer was the deciding axis. The decoder takes it on all seven tasks and on the mean, $17.2\times$ to $29.6\times$. **We therefore make the merged LoRA decoder the default artifact**, and retain the encoder as the documented alternative for the two deployments where it still wins: a p95 latency budget it meets at 20 ms and the decoder does not at 56 ms, and a 0.6 GB footprint against 3.4 GB. §8.3 states that choice as a rule. The encoder’s own held-out results, and the findings that only its presence made visible, are in §7.1 and §7.4.

6.2 Zero-shot against the commercial reference, on all seven tasks

For most of this work we compared against Jev on Banking77 alone, because it was the only task we had measured them on, and reported the result as a single twenty-one point gap. Table 3 runs their API over the whole held-out suite, 600 items per task, on the same items we score ourselves on.

Widening the comparison did not change its direction, only our confidence that the direction is general. It also moved their Banking77 number from 0.820 to 0.863; the service is neither deterministic nor static, so both are measurements at a point in time rather than properties of the product.

This is the sharpest statement of what this paper does and does not claim. **An open 1.7B model with 87 MB of adapters does not approach a commercial typed-decision API zero-shot, on any task we tried.** The case for building one rests on the other half of the result: a few hundred labels of your own task reverse the ordering, which Section 6.5 quantifies.

6.3 The output format makes zero-shot scoring possible, not present

Before any task training, we score the held-out suite using only the output format and a pretrained backbone.

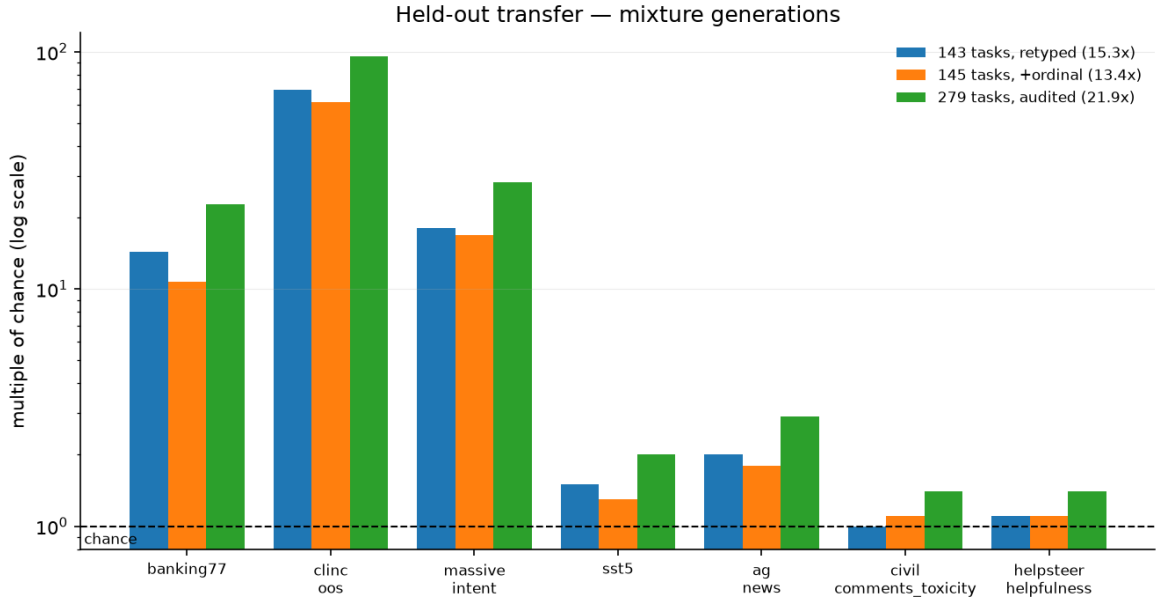


Figure 4: Held-out transfer across three mixture generations, log scale, chance marked. The middle generation is *worse* than the first: that is negative transfer from a mixture change that traded away **choice** tasks, retained here because a plot of only the best run would imply progress that did not happen.

backbone	configuration	held-out mean
decoder, 1.7B	state and options concatenated plainly	0.8× chance
decoder, 1.7B	+ each option framed as yes/no	2.9× chance
decoder, 1.7B	+ a preamble stating the task	11.5× chance
decoder, 1.7B	<i>control</i> : shortened framing	5.7× chance
encoder, 150M	pretrained masked-LM head at the marker	0.7× chance
encoder, 395M	same readout, larger model	2.2× chance

The two halves of the table are different backbones and are not directly comparable; each is internally consistent. Within the decoder rows, the format is worth an order of magnitude, and the last decoder row is the control that matters: shortening the framing to `{opt}\ncorrect? answer` roughly halves the mean, so the gain is the format doing work rather than an incidental wording choice.

Within the encoder rows, a strong bidirectional model with the same output format scores *below chance*, and its larger sibling barely clears it. This is the negative result that organises the rest of the paper. Nothing about scoring options at marker positions confers generalization, on either backbone: 11.5× from prompt framing alone is a long way from the 29.6× that training produces, and 0.7× is worse than guessing. What the format buys is a well-formed output and a cheap one, not an ability.

The same sensitivity appears in option rendering. On the held-out binary task, scoring the two `noul` descriptions rather than bare polarity costs AUROC 0.63 against 0.712, because the descriptions restate one judgement from opposite sides and a model scoring both measures their overlap.

task	K	Jev	95% CI	OpenJev	
clinc_oos	151	0.938	[0.918, 0.958]	0.702	Jev
ag_news	4	0.880	[0.853, 0.905]	0.793	Jev
banking77	77	0.863	[0.835, 0.890]	0.605	Jev
massive_intent	60	0.838	[0.807, 0.867]	0.775	level
civil_comments	2	0.748	[0.713, 0.785]	0.742	level
sst5	5	0.560	[0.522, 0.598]	0.465	Jev
helpsteer	5	0.415	[0.378, 0.455]	0.273	Jev
mean multiple of chance		38.3		29.6	Jev

Table 3: Zero-shot on schemas neither system was trained on. Jev wins five and ties two, where a tie means the intervals overlap. It does not lose one. Their per-item predictions ship in the repository so the comparison is checkable without an API key.

6.4 What produced the transfer

Label-space augmentation is the single largest effect we measured:

held-out task	K	before	after
banking77	77	0.000	0.205
clinc_oos	151	0.000	0.180
massive_intent	60	0.040	0.305
ag_news	4	0.225	0.655

Two exact zeroes become usable numbers. You do not need datasets with large menus; you can synthesize them. The caveat is that the mixture also grew from 61 to 141 tasks in the same change, so scale and shape are not separated in this pair.

Scaling the audited mixture to 279 tasks took the mean from $17.6\times$ to $17.2\times$ and moved every task (Figure 4).

Both of those measurements were made on the encoder, before the backbone was settled. We did not repeat the augmentation ablation on the decoder, so the effect size is an encoder effect size; the decoder was trained on the post-augmentation mixture throughout and we cannot say from this work how much of its $29.6\times$ the augmentation is responsible for.

6.5 Router: a fine-tuned small model against a zero-shot API

The comparison is asymmetric, in our favour, and we state it before the numbers. OpenJev is fine-tuned on 395 labelled examples of this task. The reference API is not: it has never seen this task’s training split, and produces its answers from the schema alone. This is not a like-for-like contest, and a reader should discount it accordingly. What it establishes is narrower than “we beat Jev”: *if you can label a few hundred examples, a 1.7B open model reaches and passes a closed API operating zero-shot on the same task.* That is a claim about the value of labels as much as about either model.

Two tier diagnostics that look like wins are not. Below the primary measures, two tiers show large margins and fail the significance test:

	OpenJev <i>fine-tuned</i>	reference <i>zero-shot</i>	b_{10}	b_{01}	p	winner
intent	0.979	0.941	14	1	9.8e-04	OpenJev
multi-label exact set	0.909	0.822	57	18	7.2e-06	OpenJev
pharmacy scope gate	0.978	0.880	49	5	3.9e-10	OpenJev
clinical gate	0.987	0.978	7	3	0.34	level
abusive gate	0.993	0.996	2	3	1.00	level
injection gate	0.993	0.996	1	2	1.00	level
compound exact set	1.000	0.909	2	0	0.50	level

Table 4: Rank-16 LoRA on Qwen3-1.7B, 395 training examples, 258 seconds, paired on the same 450 test items with exact McNemar. OpenJev is fine-tuned; the reference is zero-shot. Three wins, four ties, no losses across the seven primary measures.

tier	OpenJev	reference	b_{10}	b_{01}	p	n
compound_3 exact set	0.903	0.645	11	3	0.057	31
oblique clinical recall	0.966	0.793	5	0	0.063	29

We report both as level, because that is what the test says. They are the two tiers we most expected to win, which is exactly why they need the test rather than the margin, and at 31 and 29 items the oblique-clinical tier floors at $p = 0.0625$ even with zero losses. Read them as underpowered rather than as parity.

compound_3 is nonetheless where the `noul` primitive earns its place in design terms: three simultaneous intents in one message, which a single `choice` softmax cannot express at all, whatever the p -value on 31 items says.

The smallest configuration is the best one.

configuration	intent	trainable	artifact
decoder, LoRA $r=16$	0.979	17M	87 MB
decoder, all open	0.929	1,725M	3.4 GB
decoder, head only	0.666	4.2M	17 MB
encoder, all open	0.899	150M	0.6 GB

Adapting 17M parameters beats opening 1,725M, at 1/39 the artifact. The likely explanation is the learning rate a full fine-tune tolerates: 10^{-5} over six epochs of 395 examples barely moves a 1.7B model, while LoRA at 2×10^{-4} adapts quickly without disturbing the weights it rides on. Neither was tuned beyond one setting, so this is “adapters are the right default here” rather than a tuned optimum. The encoder row is the alternative backbone fully fine-tuned, which is the strongest thing we could do with it on this task, and it lands eight points below the adapter.

Intermediate-task transfer, which works on one backbone only. Starting the *encoder* specialist from a general checkpoint rather than raw weights, with identical data and schedule, moves intent $0.544 \rightarrow 0.899$ and multi-label $0.453 \rightarrow 0.789$ ($p = 6e-18$). Two safety gates that never fired at all from scratch reach 0.857 and 0.583 recall. This is the small-target-data regime where STILTs [4] predicts the largest gain, and it is one of the strongest single interventions in this work.

batch	encoder		decoder	
	states/s	p50	states/s	p50
1	47.4	20.4 ms	23.2	42.9 ms
4	188.7	20.8 ms	74.5	54.7 ms
8	373.9	21.0 ms	77.1	105.0 ms
16	654.6	24.0 ms	77.3	206.4 ms
32	532.7	36.8 ms	84.4	417.0 ms

Table 5: Sustained throughput, ten questions per state, idle H100, one process. Each state answers ten questions, so the encoder’s peak is 6,546 decisions per second against the decoder’s 844.

The same intervention hurts the decoder. The 0.979 LoRA used no mixture training at all; it started from base Qwen3-1.7B, and starting it from our general adapter instead is significantly *worse* (0.953, $p = 0.012$). §7.4 works through why. The general checkpoint earns its keep on schemas you have no labels for, not on the task you actually care about.

6.6 Throughput under concurrent load

Every latency figure in this paper is batch size 1, the interactive case, and the reciprocal of a single-request latency is not a throughput number. Table 5 sweeps batch size as a stand-in for in-flight requests, which is what a server batching arrivals sees.

The reference API peaks near 47 requests per second at 32 in-flight, with its median flat around 436 ms, and degrades past that: at 64 concurrent its median reaches 1,106 ms while throughput falls to 29.8/s. On that comparison the decoder sustains 84.4/s at a comparable 417 ms, and the encoder sustains 654.6/s at 24 ms.

Two caveats, both material. Ours is a single process on one GPU with no HTTP layer, no queuing and no network, while theirs is a hosted service measured across the internet from India, so this bounds what the hardware can do rather than what a productionised OpenJev would serve. And the encoder has a knee of its own between batch 16 and 32, where throughput falls from 654.6 to 532.7 and p95 rises from 29.4 to 209.1 ms, the same character as the reference system’s knee at a different position.

This is the strongest case for the encoder in this paper. It was already the choice when p95 latency binds. It also serves roughly eight times the decoder’s throughput, and that gap is larger than the accuracy gap running the other way. A deployment answering millions of decisions a day should read Table 5 before Table 2.

6.7 Latency and footprint

Two measurements, taken under different load, so the absolute values are not comparable across them. The first is a back-to-back comparison of the three serving configurations while a training run shared the GPU, which inflates every row equally and leaves the ratios readable:

	p50	p95	vs encoder
encoder, 150M	37.7 ms	50.9 ms	baseline
decoder LoRA, unmerged	74.0 ms	110.5 ms	1.96×
decoder LoRA, merged	40.4 ms	55.9 ms	1.07×

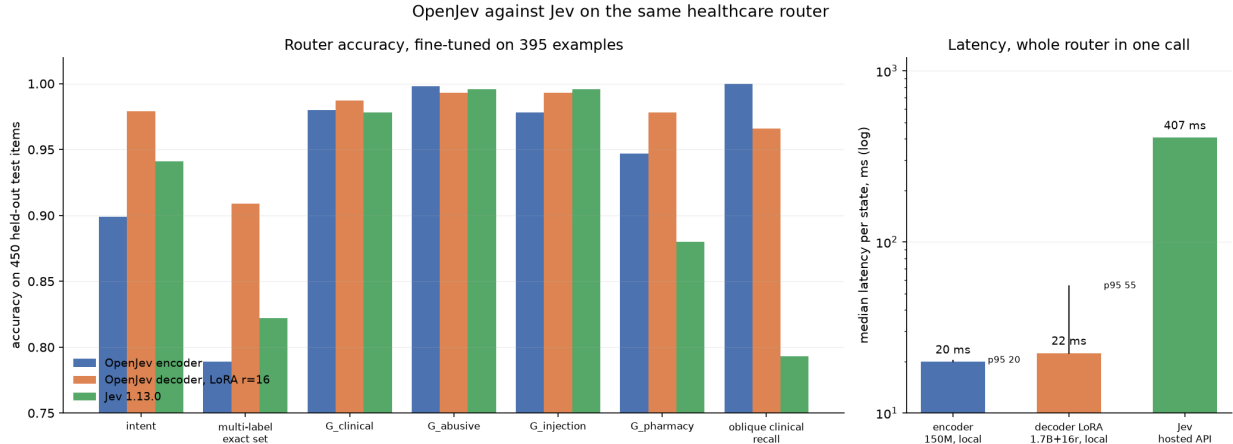


Figure 5: OpenJev against the reference API on the router. Accuracy left, latency right. The latency panel is not like-for-like: our figures are local GPU compute and the reference is an end-to-end hosted call including network, so it is the latency a user experiences rather than a claim about their model’s speed.

An unmerged adapter costs roughly $2\times$ for nothing, since it is an extra matmul per target module at every layer and the output is identical either way. Merged, a model eleven times larger runs 7% slower. The released loader therefore merges on construction (§3.7), so this cost cannot be paid by accident.

The second measurement is the fine-tuned router at batch size 1, ten questions and twenty-four options per state, under lighter load. It is the one the deployment recommendation rests on, because the tail is what a latency budget is written against:

	decoder, 1,725M	encoder, 150M
p95 per state	56 ms	20 ms
checkpoint on disk	3.4 GB	0.6 GB
adapter, per additional task	87 MB	0.6 GB

The tail, not the median, is where the two backbones separate, and it is the only axis on which the encoder is clearly ahead.

The two measurements disagree about how much, and we have not settled it. Under GPU contention the merged decoder’s p95 is $1.1\times$ the encoder’s (55.9 against 50.9 ms); on the lighter-load router it is closer to $2.8\times$ (56 against 20 ms). The likely reading is that contention inflates the encoder’s tail far more than the decoder’s in relative terms, since both are sitting on the same queue, so the lighter-load figures are the ones a dedicated deployment would see. We did not re-run the merge comparison on an idle GPU to confirm that, so anyone with a tail budget in the tens of milliseconds should measure on their own hardware rather than take either ratio from us. Everything here is batch size 1, which is the interactive case; sustained throughput under load is not measured in this work.

6.8 Benchmark transfer is not deployment readiness

The general decoder scores $29.6\times$ chance on the held-out suite and the general encoder $17.2\times$. Applied zero-shot to the routing task, that ordering reverses:

	decoder <i>zero-shot</i>	encoder <i>zero-shot</i>	reference <i>zero-shot</i>
intent	0.467	0.601	0.941
clinical gate recall	0.380	0.111	0.926

The decoder wins all seven public benchmarks and is worse than the encoder on intent for the one task with an operational shape: multi-question packing, compound multi-label answers, and gates with recall floors. Neither backbone is usable there. Intent at 0.601 on a six-way menu is genuinely above chance, so some transfer is real, but a clinical gate firing on one in nine of the cases it should catch is not a gate, and 0.380 is not one either. The decoder’s other zero-shot figures on this task are no better: multi-label exact set 0.207, and a pharmacy scope gate at 0.440 recall with a 0.696 false-positive rate. Fine-tuning on 395 examples of the task takes the same decoder to intent 0.979 and multi-label exact set 0.909 (Table 4), so what is missing zero-shot is task supervision rather than capacity.

We report this because it undercuts the natural reading of our own headline result. It does not change the architecture decision, which rests on zero-shot transfer where no labels exist and on fine-tuned accuracy where they do, and the decoder wins both of those. It does mean that clearing chance on academic benchmarks with clean label sets is a materially weaker claim than working on a deployment task you have no labels for, that the two come apart sharply here, and that nobody should read $29.6\times$ as a statement about a system anyone would ship unsupervised. Safety gates in particular appear to require task supervision.

7 Ablations, negative results, and one retraction

Reporting only what worked would misrepresent both the difficulty and the method. This section holds the backbone we did not choose and the evidence it produced, three interventions that sounded right and did nothing or actively hurt, and one conclusion we published internally and then withdrew.

7.1 The encoder backbone: the alternative, and the control

The bidirectional encoder was the first backbone we built and it is no longer the recommended one. It is kept for two reasons. It is a released artifact with a real deployment case (§8.3), and it is the control that makes several results in this paper legible: an intervention that helps one backbone and hurts the other cannot be seen with one backbone alone.

Table 6 is the encoder trained on the same audited 279-task mixture, with the same augmentation, evaluated on the same items at full menu size. Harness sanity 1.000; held-in control 1.1 to $2.3\times$ chance.

This is a real result and we do not withdraw it: all seven tasks clear their chance floor and a 151-way menu reaches $57.6\times$. It is also the weaker of the two backbones on every task, and one task is weaker than the table implies. Against the majority-class baseline rather than $1/K$, `civil.comments` wins outright (0.683 against 0.500, $p < 10^{-15}$) but `helpsteer` sits on the line: 0.262 against a 0.233 baseline, $p = 0.050$. Six of seven are unambiguous; the seventh beats chance while

task	K	chance	accuracy	95% CI	$\times$ chance
clinc_oos	151	0.007	0.382	[0.342, 0.420]	57.6
massive_intent	60	0.017	0.473	[0.430, 0.512]	28.4
banking77	77	0.013	0.343	[0.305, 0.382]	26.4
ag_news	4	0.250	0.735	[0.698, 0.772]	2.9
sst5	5	0.200	0.412	[0.373, 0.452]	2.1
civil_comments	2	0.500	0.683	[0.648, 0.718]	1.4
helpsteer	5	0.200	0.262	[0.225, 0.297]	1.3
mean multiple of chance					17.2

Table 6: Held-out transfer for the encoder, 600 rows per task, never trained on, every task scored at its full advertised menu. This is the comparison column of Table 2 in full. Every interval excludes its chance floor, and the suite began this work at $0.9\times$ chance. An earlier draft reported $21.9\times$ here from an in-training evaluator run against a superseded version of the suite; these figures come from `eval_heldout.py` on the published checkpoint and the published suite, and reproduce from a clean clone.

only matching the trivial baseline. The decoder removes that caveat, though its own **helpsteer** margin is thin ($p = 0.010$).

Three findings the encoder is responsible for. None of these survives if you only ever train one backbone.

- **The output format confers nothing by itself.** Untrained, with the option-scoring readout and no task training, the encoder scores $0.7\times$ chance, worse than guessing, and its 395M sibling reaches $2.2\times$ (§6.3). An untrained decoder reads menus at $11.5\times$, but only because it borrows a pretrained vocabulary head, which is a property of the backbone and not of the architecture. The encoder is what separates those two explanations.
- **Intermediate-task transfer reverses by backbone.** +36 points on the encoder and significantly negative on the decoder, from the identical intervention (§7.4).
- **Benchmark transfer does not rank models on deployment.** The decoder beats the encoder on all seven public tasks and is worse than it zero-shot on the router (§6.8). With one backbone this would have read as a flat statement that zero-shot routing is hard, rather than as a reversal.

7.2 Interventions that failed

Ordinal scale augmentation: null. The mixture’s **score** tasks are mostly three-level sentiment scales, against a held-out task wanting a five-level judgement. We synthesized the missing diversity the way label-space augmentation synthesized menu size: merging adjacent levels and remapping the gold, and restating a scale in another vocabulary of the same length within its semantic family. Ablated at $\pi_{\text{scale}} \in \{0, 0.5\}$ with everything else fixed:

	$\pi_{\text{scale}}=0$	$\pi_{\text{scale}}=0.5$
helpsteer	0.110 (0.6 $\times$)	0.128 (0.6 $\times$)
sst5	0.342 (1.7 $\times$)	0.348 (1.7 $\times$)
suite mean	16.5$\times$	15.7 $\times$

Both ordinal tasks land on the same multiple of chance either way, and the mean is slightly *worse* with it. The mechanism is correct and tested (golds stay on the right level, merges stay contiguous and monotone) it simply is not the bottleneck.

Adding a primitive by removing another: negative, twice. The mixture was poor in **noul** (13 tasks) and **score**. We recovered 21 real yes/no tasks by recognising negation pairs (**not_paraphrase** vs **paraphrase**) that the builder had been emitting as two-option menus, and separately added two real ordinal datasets. Both made things worse:

mixture	choice tasks	banking77	suite mean
143 tasks, baseline	121	0.278	17.6$\times$
143 tasks, 21 retyped to noul	100	0.187	15.3 $\times$
145 tasks, + 2 ordinal	100	0.138	13.4 $\times$

banking77 tracks the **choice** task count exactly. Five of seven held-out tasks are **choice**, so converting or diluting **choice** tasks is a trade rather than an improvement. This is textbook negative transfer [4], and we caused it twice before measuring it. The fix was scale: the audited 279-task mixture restored **choice** to 234 and added tasks on top, reaching 17.2 $\times$.

7.3 A conclusion we retracted

For several runs we reported that the **noul** primitive does not transfer, on the evidence that the held-out binary task sat at exactly 0.500 accuracy with macro-F1 0.333, which is a model answering the same way every time. We spent two separate interventions on it.

Both were aimed at the wrong thing. The model ranks that task at AUROC 0.714 and reaches 0.680 accuracy at a fitted threshold; it puts 98% of its probability mass above 0.5, so **argmax** calls everything positive and accuracy lands on the class balance. **That is a calibration result, not a transfer result.** Reporting **argmax** accuracy alone on an uncalibrated binary measures the threshold as much as the model, and our evaluation now reports AUROC alongside it.

Two measurement errors of our own contributed, and both are worth stating because they are easy to repeat:

- **A label inversion in the inference helper.** Option names were derived in *dictionary* order, so a **noul** whose criteria were written `{true: ..., false: ...}` had its two probabilities swapped. This turned AUROC 0.712 into 0.288, exactly $1 - 0.712$, and looked like a model ranking toxic comments as cleaner than clean ones. The giveaway was the arithmetic.
- **A partially loaded model that produced a full results table.** A decoder was built without its trained head, so `load_state_dict(strict=False)` silently dropped six tensors and scored the untrained readout, reporting 5.8 $\times$ chance where the truth was 16.2 $\times$. A warning printed; a complete results table printed under it. Loaders now raise.

We also credited the held-in control with catching the second bug, and that was wrong too: the decoder reads 1.0 to 1.3 $\times$ on held-in tasks even when it loads perfectly, because a 4.2M head on a frozen backbone barely fits its own training mixture. Only the harness sanity task caught it.

7.4 Intermediate-task transfer helps the encoder and hurts the decoder

The clearest illustration that an intervention is not good in itself, only good relative to what the base model already does. Initialising the router adapter from the general adapter of §6.1, rather than from base Qwen3, and training identically:

initialisation	intent accuracy	95% CI
base Qwen3	0.979	[0.962, 0.994]
general adapter	0.953	[0.926, 0.973]

Paired McNemar on the same 338 items: $p = 0.012$, with 10 items the base initialisation gets right that the general one misses against 1 in the other direction. The general initialisation is significantly *worse*.

On the encoder path the identical intervention was worth +36 points. The contradiction resolves on what each backbone lacked. The encoder had to learn the very notion of scoring against a menu from 395 router examples, so the mixture supplied a missing capability. Base Qwen3 already reads menus from pretraining, so the mixture supplies nothing new and its 279-task specialisation acts as interference on a narrow fourteen-intent problem. This is negative transfer, the second instance measured in this work.

The practical rule we draw: stage your fine-tuning when the base model cannot perform the task *form* at all, and train the task adapter directly from base once it can. The general adapter remains the right artifact for zero-shot use, where by definition no task data exists to train on; it is the wrong starting point when task data does exist.

7.5 Interventions that worked

For completeness, the positive results, ordered by effect size: label-space augmentation (§6.4); the decoder backbone over the encoder ($17.2\times \rightarrow 29.6\times$, §6.1); mixture scale and auditing ($17.6\times \rightarrow 17.2\times$); prompt format on a decoder backbone ($0.8\times \rightarrow 11.5\times$); intermediate-task transfer on the encoder only (+36 points at 395 target examples, and see §7.4 for where it reverses); LoRA over full fine-tuning (0.979 vs 0.929 at 1/39 the size); and adapter merging ($1.96\times \rightarrow 1.07\times$ latency).

7.6 Methodological caveat

One choice in this paper was made badly. The `noul` rendering rule (§3) was selected by comparing variants on the held-out task itself, which is the wrong place to choose anything. The change is principled and small, and the three-way spread we observed across connective phrasings (0.25 to 0.75 AUROC) is large enough that it should be re-settled on held-in data before the held-out number is quoted as clean.

8 Discussion

8.1 What the evidence supports

The split is clean and it is about labels. **Given roughly 400 labelled examples for your task, a 1.7B model with an 87MB adapter beats a commercial typed-decision API** on a hardened routing benchmark, runs on your own hardware, returns full-precision probabilities, and is deterministic. **Without labels, it is behind on every task we measured:** across the seven held-out schemas Jev wins five and ties two, $38.3\times$ chance against our $29.6\times$, with 0.863 against

0.605 on Banking77 and 0.938 against 0.702 on the 151-way menu. We compared on Banking77 alone for most of this work and called it a twenty-one point gap; measuring the rest of the suite did not change the direction of the result, only our confidence that it is general. On the encoder Banking77 was 0.343, so changing the backbone closed part of that gap without new data and left the rest open.

Both halves matter. The first says the capability is not gated behind scale or a vendor. The second says the vendor’s actual product, working on a schema you have never labelled, is a real and unsolved thing.

8.2 When this fits a real problem

Published walkthroughs of the reference API give a useful external check on whether our primitive set matches real demand. Three that circulated widely: filtering 38,000 LinkedIn connections against an ideal-customer profile, cleaning 4,710 auto-collected Gmail contacts, and triaging six months of arXiv abstracts for relevance to non-technical readers.

All three use only **choice**, **score** and **noul**; all three ask several questions per record in one call; all three threshold on the returned confidence. Menu sizes run from 4 to 15, well inside what we handle. So the abstraction transfers, which is worth stating because we did not design it and it could have turned out narrower than the real demand.

Whether *our* implementation fits depends on one thing:

	no labels	a few hundred labels
schema you have never seen	reference API	OpenJev
cost per record at scale	comparable	OpenJev (local)
need a real probability	OpenJev	OpenJev
need reproducibility	OpenJev	OpenJev

The probability and reproducibility rows are ours regardless of labels: the reference quantises to a 0.01 grid with 71.9% of values at a hard zero, which matters directly for the confidence thresholds all three of those applications expose to the user, and it offers no seed.

Two practical observations. First, on our own evidence a task dominated by **score** questions is the worst case for OpenJev, since the ordinal primitive is where we only match a majority baseline; two of the three walkthroughs contain such questions. Second, and more usefully: each of those three workflows *already* contains a labelling step, a 40-row corrected evaluation file in one and a 50-item checkpoint in another, used for validation rather than training. The labels a fine-tune needs are already being produced. Raising 40 to 300 converts a validation artifact into a training set, and on our measurements that is the difference between trailing the reference API and passing it.

8.3 Which backbone to use

We ship both backbones, so the recommendation needs to be concrete rather than a shrug. **Use the merged LoRA decoder unless one of two specific constraints binds.**

	decoder, merged LoRA	encoder, fine-tuned	source
held-out suite mean	29.6×	17.2×	§6.1
clinc_oos, $K=151$	0.702	0.382	§6.1
Banking77 zero-shot	0.605	0.343	§6.1
router intent, fine-tuned	0.979	0.899	§6.5
p95 per state, batch 1	56 ms	20 ms	§6.7
checkpoint on disk	3.4 GB	0.6 GB	§6.7
cost of a second task	87 MB adapter	0.6 GB checkpoint	§6.5

Take the decoder by default. It is better on all seven held-out tasks, better by eight points of intent accuracy on the fine-tuned router, and the merged adapter costs $1.07\times$ the encoder’s median latency, so the size difference mostly does not show up in wall time at batch 1. Serving many schemas from one backbone is also cheaper on this path, since each additional task is an 87 MB adapter rather than another 0.6 GB of weights.

Take the encoder if the p95 tail is your budget. The median latencies are close; the tails are not, at 56 ms against 20 ms on the router measurement. If your router sits on a request path with a hard tail budget in the tens of milliseconds, that is the measurement that decides it, and no amount of accuracy recovers a missed deadline. This is the weakest recommendation in this subsection: our two latency runs disagree on the size of the tail gap ($1.1\times$ under GPU contention against $2.8\times$ under lighter load, §6.7), and we did not resolve which is representative. Measure it on your own hardware before letting it decide.

Take the encoder if 3.4 GB does not fit. A 0.6 GB checkpoint runs on hardware a 3.4 GB one does not, including CPU-only and small-GPU deployments we did not benchmark. This is a hard constraint rather than a trade, and where it binds the encoder’s $17.2\times$ is still a working system.

Two things that should not enter the decision. Do not leave the adapter unmerged to keep the artifact separate at serving time: it costs $1.96\times$ latency for output that is identical either way (§6.7). And do not pick the backbone on held-out benchmark transfer if your real task looks like our router, because that ordering reverses (§6.8). If you have labels, fine-tune both and measure; it took 258 seconds on the decoder and 38 on the encoder.

8.4 Why the data mattered more than the model

The most useful finding here is negative: a strong encoder with a clever output format scores below chance untrained, and an untrained decoder with the same format reaches only $11.5\times$ against the $29.6\times$ that training produces. Nothing about scoring options at marker positions confers generalization on either backbone. What produced it was the composition of the training mixture, and in particular a property we could manufacture, namely menu-size diversity, rather than one we had to find. The backbone swap is worth $17.2\times$ to $29.6\times$ on top of that data work, which is a large effect and still the second-largest one in this paper.

This has a practical consequence. Our mixture is 234 **choice** tasks against 34 **noul** and 11 **score**, and the result pattern follows exactly: **choice** transfers, the other two barely do. That is a statement about 279 tasks scraped from one source, not about the architecture. The fastest route to a better general model is a more balanced typed-decision corpus, and assembling one is beyond what a single project can do.

8.5 What the zero-shot gap does and does not show

Jev wins five of the seven held-out tasks and ties two (Section 6.2). It is worth being precise about what that licenses, because the obvious reading, that the recipe does not work, is not the reading the evidence supports.

Every intervention that added data or data diversity improved generalization, and none of them plateaued. A 61-task mixture transferred nothing measurable, $0.9\times$ chance. Padding training menus with borrowed distractor labels moved a 77-way held-out menu from a literal 0.000 to 0.205. Scaling and auditing to 279 tasks and 323,466 rows produced the figures in Table 2. Three interventions, three improvements, no sign of diminishing returns at the point we stopped. The shape of that curve is the central piece of evidence for the method, and it is more informative than any single accuracy number in this paper.

We stopped for reasons of budget rather than of evidence: one researcher, one GPU, and a corpus assembled from public datasets that is one to two orders of magnitude smaller than the instruction mixtures that motivated this design.

The honest limit on the argument is that **Jev is a closed API and we cannot see what it was trained on**. Its architecture, corpus and scale are all unobservable. We therefore cannot demonstrate that the remaining gap is a data-scale gap rather than a method they have and we do not. Our own scaling curve points at data, and nothing we measured indicates the architecture is the binding constraint, but the reader should treat that as the most economical explanation of our results rather than as a demonstrated fact. Distinguishing the two would need either their training details or a run at a scale we could not afford.

What the paper does demonstrate, without needing to resolve that question, is the conditional claim: given a few hundred labelled examples of a target task, the same recipe produces a model that beats the commercial system on that task, three wins and four ties against a system that is better than ours on every held-out benchmark. The recipe is sound; the general model is underfunded.

8.6 Limitations

The ordinal primitive is weak. `helpsteer` beats chance but only draws level with predicting the most common label. The mixture has 11 `score` tasks, mostly three-level sentiment. We searched twice for more; ordinal scales with five or more levels, in domains that are not response quality, are scarce in accessible form. We deliberately excluded LLM-response-rating corpora because they are the held-out task in all but name, and training on a near-twin would move the number without moving the ability.

No ordinal or rejection head. `score` is a softmax over levels; we use the ordering only in rendering and in an expectation-based readout. Rank-consistent objectives [7] and adaptive rejection boundaries [12] are the obvious next step and are not implemented.

Zero-shot on a real task is much worse than on benchmarks, and benchmark transfer does not rank models on it. The decoder’s $29.6\times$ chance on the held-out suite coexists with intent 0.467 on the routing task, a multi-label exact set of 0.207, and a pharmacy gate at 0.440 recall with a 0.696 false-positive rate. Safety gates in particular appear to need task supervision, and we would not deploy any zero-shot gate on this evidence.

The stronger version of this caveat is the comparison with the encoder. The decoder transfers far better on the benchmark suite ($29.6\times$ against $17.2\times$, every task improved) and yet is *worse* than the encoder zero-shot on the routing task, intent 0.467 against 0.601. Its clinical-gate recall

is the better of the two (0.380 against 0.111) and still nowhere near usable, so this is a reversal between two unusable systems rather than a case for the encoder.

The ordering that held across seven public benchmarks reverses on the one task with an operational shape: multi-question packing, compound multi-label answers, and gates with recall floors. We report this because it undercuts the natural reading of our own headline result. Held-out benchmark transfer measured what we asked it to measure and did not predict deployment readiness, and we would not have discovered that by running the suite alone. It does not change the architecture choice, which rests on zero-shot transfer where no labels exist and on fine-tuned accuracy where they do, and the decoder wins both; it does mean nobody should read $29.6\times$ as a statement about a system anyone would ship unsupervised.

Synthetic evaluation. The routing task is handwritten and systematically composed, not real traffic. It is evidence a method works, not a claim about any production distribution.

One-day black-box baseline. All reference figures are measurements of a hosted endpoint on a single day from one network location.

Untuned comparisons. The LoRA-versus-full-fine-tune result compares one hyperparameter setting each. We believe the direction and not the magnitude.

Single runs, no seed variance. Every number in this paper is one training run. The confidence intervals are bootstrap over the test set, which captures sampling noise but says nothing about run-to-run variance from initialisation, data order, or augmentation sampling. Differences of one or two points between configurations should not be read as real on this evidence. The large effects (a literal 0.000 becoming 0.205; 0.666 against 0.979) are far outside any plausible seed noise; the small ones are not.

The obvious baseline is missing. We do not compare against m independent single-question classifiers, which is the natural alternative to packing and the first thing a careful engineer would try. Packing’s advantage should be cost rather than accuracy, since separate classifiers have strictly more capacity per question, and our claim is about answering ten questions for the price of one. But we did not measure it, so the accuracy side of that trade is unquantified here. Nor do we compare against a general-purpose LLM with constrained decoding, which is what most teams would reach for today.

Two tiers we expected to win are statistically level. The `compound_3` tier and oblique clinical recall show large margins (0.903 vs 0.645 and 0.966 vs 0.793) at $p = 0.057$ and $p = 0.063$, on 31 and 29 items. They are reported as ties. A larger version of those tiers is the cheapest way to settle whether the margins are real.

8.7 Future work

The experiment that would most change these conclusions is a balanced typed-decision corpus with real `score` and `noul` diversity at the scale Flan suggests. Beyond that: adapter composition, where a task adapter is initialised from a general adapter, which is the parameter-efficient analogue of the +36-point intermediate-task result; rank-consistent ordinal heads; and calibrated rejection for out-of-scope inputs.

8.8 Conclusion

Typed decisions do not need a large model, and they do not need generation. They need the right output format, an attention mask that keeps questions independent, and above all training data whose *shape* matches deployment. Of those three, only the last was hard, and it is the one that transfers least from the literature to practice.

The system we recommend is a 1.7B decoder with a rank-16 adapter merged into it, which is small by current standards and large by the standards of the encoder we started from. The encoder is not a failed branch: it holds two deployment cases that the decoder does not, and three of the findings above exist only because we trained both and compared them. The version of this work with one backbone would have been shorter, cheaper, and considerably less informative.

We did not match the commercial system zero-shot, and we report that on seven tasks rather than the one we could have chosen. We also did not find the ceiling of the approach: every time we added data the numbers improved, and they were still improving when the budget ran out. Whether the remaining gap is scale or something the vendor knows that we do not is a question a closed API does not let us answer. What this work does settle is narrower and, for most people with a classification problem, more useful: the recipe is public, it runs on one GPU, and given a few hundred labels of your own it beats the commercial system on your task.

References

- [1] Suchin Gururangan, Ana Marasović, Swabha Swayamdipta, Kyle Lo, Iz Beltagy, Doug Downey, and Noah A. Smith. Don’t stop pretraining: Adapt language models to domains and tasks. In *ACL*, 2020.
- [2] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *ICLR*, 2022.
- [3] Shayne Longpre, Le Hou, Tu Vu, Albert Webson, Hyung Won Chung, Yi Tay, Denny Zhou, Quoc V. Le, Barret Zoph, Jason Wei, and Adam Roberts. The flan collection: Designing data and methods for effective instruction tuning. In *ICML*, 2023.
- [4] Jason Phang, Thibault Févry, and Samuel R. Bowman. Sentence encoders on STILTs: Supplementary training on intermediate labeled-data tasks. *arXiv preprint arXiv:1811.01088*, 2018.
- [5] Nir Ratner, Yoav Levine, Yonatan Belinkov, Ori Ram, Inbal Magar, Omri Abend, Ehud Karpas, Amnon Shashua, Kevin Leyton-Brown, and Yoav Shoham. Parallel context windows for large language models. In *ACL*, 2023.
- [6] Victor Sanh, Albert Webson, Colin Raffel, Stephen H. Bach, et al. Multitask prompted training enables zero-shot task generalization. In *ICLR*, 2022.
- [7] Xintong Shi, Wenzhi Cao, and Sebastian Raschka. Deep neural networks for rank-consistent ordinal regression based on conditional probabilities. *arXiv preprint arXiv:2111.08851*, 2021.
- [8] Damien Sileo. tasksource: A dataset harmonization framework for streamlined nlp multi-task learning and evaluation. In *LREC-COLING*, 2024.

- [9] Benjamin Warner, Antoine Chaffin, Benjamin Clavié, Orion Roberts, Ramesh Nallapati, et al. Smarter, better, faster, longer: A modern bidirectional encoder for fast, memory efficient, and long context finetuning and inference. *arXiv preprint arXiv:2412.13663*, 2024.
- [10] An Yang, Anfeng Li, Baosong Yang, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- [11] Ping Yang, Junjie Wang, Ruyi Gan, Xinyu Zhu, Lin Zhang, Ziwei Wu, Xinyu Gao, Jiaxing Zhang, and Tetsuya Sakai. Zero-shot learners for natural language understanding via a unified multiple choice perspective. In *Findings of EMNLP*, 2022.
- [12] Hanlei Zhang, Hua Xu, and Ting-En Lin. Deep open intent classification with adaptive decision boundary. In *AAAI*, 2021.

A Terminology for readers new to the area

This appendix defines every term the paper uses, assuming no background. Nothing here is needed by a reader who already works on language models; it exists so the main text can stay short.

A.1 The problem

State

The input being judged: a customer message, a product review, a comment. One state can have many questions asked about it.

Question

One decision to make about the state, with instructions and a set of candidate answers.

Option, menu, criteria

Used interchangeably for the candidate answers of a question. A *fixed menu* is shared by every row of a task; a *per-example menu* differs row by row, as in reading comprehension where each question has its own answers.

K

The number of options. $K=151$ means a 151-way choice.

Primitive

The type of a question: **choice** (pick one of K), **score** (pick one level on an ordered scale), **noul** (yes or no).

Multi-label

When more than one label can be true at once. A single **choice** cannot express this, because its probabilities sum to one across options. Several **noul** questions can.

A.2 How models process text

Token

A chunk of text, roughly a word piece. Models read and write sequences of tokens, not characters.

Embedding, hidden state

The vector a model holds for each token position after processing. The hidden state at a position is what we read a decision from.

Attention

The mechanism by which a token’s representation is built from other tokens. An *attention mask* says which positions may look at which.

Encoder vs decoder

Two attention patterns. An **encoder** is bidirectional: every token sees every other. A **decoder** is causal: a token sees only what came before it, which is what makes text generation possible. **In this paper “decoder” names the attention pattern only.** We never generate text; we run one forward pass and read scores. The decoder is the backbone we recommend; the encoder is the documented alternative (§8.3).

Forward pass

One run of the model over an input. Our central efficiency claim is that m questions cost one forward pass, not m .

Logit

A raw, unnormalised score. Logits become probabilities through a softmax.

Softmax

Turns a vector of logits into probabilities that sum to one. Our *grouped* softmax does this separately within each question’s options, because questions in one packed sequence have different K .

Backbone

The pretrained model we build on, before any of our training.

A.3 Training

Pretraining

The expensive first stage, done by others, where a model learns language from large text corpora. We never do this.

Fine-tuning

Further training of a pretrained model on a specific task. Cheap by comparison: 38 seconds in one of our settings.

Epoch

One pass over the training data.

Learning rate

How large a step training takes per update. It matters more than it sounds: our LoRA-beats-full-fine-tuning result is most likely a learning-rate effect (§6.5).

Frozen

A parameter that is not updated during training.

LoRA (Low-Rank Adaptation)

Instead of updating a large weight matrix, train two small matrices alongside it and add their product. The large matrix stays frozen. The **rank** r sets how small; $r=16$ here. The result is an **adapter** of tens of megabytes rather than gigabytes, so one backbone can serve many tasks.

Merging

Folding a trained adapter back into the frozen weights, so inference costs no extra work. An unmerged adapter is roughly twice as slow for no benefit (§6.7).

Multi-task mixture

Training on many tasks at once, so the model learns the general shape of the problem rather than one instance of it.

Intermediate-task transfer

Train on task A, then on target task B. Helps most when B is small. Known in the literature as STILTs [4].

Negative transfer

When adding training data makes the target *worse*. We caused this twice (§7).

A.4 Evaluation

Zero-shot

Evaluating on a task the model was never trained on. The hard claim, and the thing Jev is good at.

Held-out

Data deliberately excluded from training, used to measure zero-shot ability. Worthless if it leaks into training, hence the contamination guard.

Held-in control

Accuracy on tasks the model *was* trained on, reported next to held-out. If held-out is at chance and held-in is high, the model truly fails to transfer; if both are at chance, something is broken. Without this, a bug and a finding look identical.

Chance

What random guessing scores: $1/K$. On a 151-way menu that is 0.007, so an accuracy of 0.29 is strong; on a 4-way menu chance is 0.25 and 0.29 is nearly worthless.

Multiple of chance

Accuracy divided by chance, so tasks with very different K can be compared on one axis.

Majority-class baseline

Always predicting the most common label. On skewed data this beats chance, so it is the stricter floor. One of our encoder results passes the chance test and only ties this one (§7.1).

Accuracy, precision, recall

Accuracy is the fraction correct. *Recall* is the fraction of true positives found, which is what matters for a safety gate. *Precision* is the fraction of positive predictions that were right.

Macro-F1

The harmonic mean of precision and recall, averaged over classes so rare classes count as much as common ones. A binary task scoring macro-F1 0.333 is predicting one class for everything.

AUROC

Ranking quality, independent of any threshold: the probability a random positive is scored above a random negative. 0.5 is chance. It matters because a model can rank well and still score badly at a fixed threshold, which is a calibration problem and not a capability one.

Calibration

Whether a stated probability matches reality. If everything a model calls 0.7 is right 70% of the time, it is calibrated. **ECE** measures the gap; **temperature scaling** is the standard post-hoc fix.

Bootstrap confidence interval

Resample the test set with replacement many times and report the middle 95% of the resulting scores. An interval that excludes chance means the result is unlikely to be luck.

McNemar test

For comparing two systems on the *same* items. Counts only the disagreements: b_{10} where we are right and they are not, b_{01} the reverse, and asks whether the split is lopsided enough to be real. Necessary because two systems can have equal accuracy while disagreeing on a third of the set.

***p*-value**

Informally, the probability of seeing a difference this large if there were no real difference. Below 0.05 is conventionally called significant. With small samples, significance can be unreachable no matter how good the model: one of our tiers has 29 items, so even a perfect score floors at $p = 0.0625$.

A.5 Terms specific to this work

Packing

Writing the state once, then all questions and options after it, as one sequence for one forward pass.

Shared prefix

The state portion of a packed sequence, attended to by every question. The reason m questions are nearly free.

Marker

A token after each option where its score is read.

Block-diagonal attention

The mask that lets each question see the shared prefix and itself, but not other questions, so an answer does not depend on what else was asked.

Label-space augmentation

Padding a training menu with labels borrowed from other tasks. The gold stays correct, but the model must discriminate against a larger menu. Our largest single effect.

Distractor

One of those borrowed labels.

Contamination guard

A check that no evaluation dataset appears in the training mixture under any of its names.

B Released artifacts

All public and Apache 2.0 unless the underlying data says otherwise.

Code. <https://github.com/S1LV3RJ1NX/openjev>

Models. All on the Hugging Face Hub.

artifact	size	link
General LoRA adapter (Study G)	87 MB	s1lv3rj1nx/openjev-general-lora
Router LoRA adapter (Study S)	87 MB	s1lv3rj1nx/openjev-router-lora
General encoder checkpoint	0.6 GB	s1lv3rj1nx/openjev-encoder-general
Router encoder specialist	0.6 GB	s1lv3rj1nx/openjev-router-healthcare

The two adapters are the recommended artifacts and both require **Qwen/Qwen3-1.7B** as their base. The two encoder checkpoints are the alternative backbone (§8.3) and are standalone.

Datasets.

artifact	size	link
Training mixture	279 tasks, 323,466 rows	s1lv3rj1nx/openjev-mixture
Held-out suite	7 tasks, 4,200 rows	s1lv3rj1nx/openjev-heldout
Healthcare router	988 items, 18 tiers	s1lv3rj1nx/openjev-healthcare-router

Base models (not ours): **Qwen/Qwen3-1.7B** [10] and **answerdotai/ModernBERT-base** [9]. The mixture is assembled from **tasksource** [8].

Loading a model. The adapter is merged into the base weights on load, so no extra inference cost is paid by accident.

```
from openjev import DecisionModel, Choice, Noul
model = DecisionModel.from_pretrained("s1lv3rj1nx/openjev-router-lora")
out = model.answer(state, {"intent": Choice(...), "clinical": Noul(...)})
out["intent"].label, out["intent"].probabilities
```

C Reproducing the main results

result	command
Held-out transfer	<code>scripts/eval_heldout.py --ckpt <ckpt></code>
Router against reference	<code>scripts/eval_router.py --ckpt <ckpt> --dump o.json</code>
Paired McNemar	<code>scripts/compare_to_jev.py --ours o.json</code> <code> --jev baselines/jev_healthcare_router.json</code>
Latency	<code>scripts/bench_latency.py --ckpt <ckpt></code>
Data audit	<code>scripts/audit_data.py <task dir> --deep</code>
Packing check	<code>scripts/check_packing.py <mixture></code>
Binary diagnosis	<code>scripts/diagnose_binary.py --ckpt <ckpt></code>

The reference system’s per-item predictions on all 450 router test items ship in the repository as **baselines/jev_healthcare_router.json**. A McNemar test needs both systems’ answers on the same items, so publishing only aggregate accuracies would let a reader check our deltas but not our significance claims. That file is one pass against a non-deterministic and evolving service, so it is the baseline we measured rather than a fixed property of the product.

D Hyperparameters

	Study G (mixture)	Study S (router)
epochs	1	6
batch size	8 dec / 16 enc	4 dec / 8 enc
context	2048	3072 dec / 2048 enc
learning rate	3×10^{-5} enc	3×10^{-5} enc, 2×10^{-4} LoRA
label-space aug. π	0.5, cap 120	off
noul recast π	0.15	off
LoRA	$r=16$, $\alpha=32$, dropout 0.05	same
LoRA targets	q,k,v,o,gate,up,down projections	

Temperature scaling is fitted on a dev split after training and stored with the checkpoint.

E Held-out suite composition

task	primitive	K	source
clinc_oos	choice	151	clinc/clinc_oos
banking77	choice	77	PolyAI/banking77
massive_intent	choice	60	AmazonScience/massive
ag_news	choice	4	fancyzhx/ag_news
sst5	score	5	SetFit/sst5
helpsteer_helpfulness	score	5	nvidia/HelpSteer
civil_comments_toxicity	noul	2	google/civil_comments

Each task declares every alias of its source; the union is the contamination manifest checked before every training run.